

5SAFE PROJECT

March 28, 2025 - July 8, 2025



Agenda

1. Introduction
2. Timeline
3. Base Idea/ System Overview
4. Hardware Setup
5. First Test
6. Development Workflow & Tools
7. Python System Architecture
8. Annotation
9. AI Model Training
10. Dashboard
11. Project Result
12. Live Demo
13. Next Steps

1. Introduction

Who we are?

Team members:

Zahra A. Computer Science

Gresë M. Computer Science

Jonas S. Electrical Engineering and Information Technology

Sebastian K. Electrical Engineering and Information Technology

General information

5-Safe:

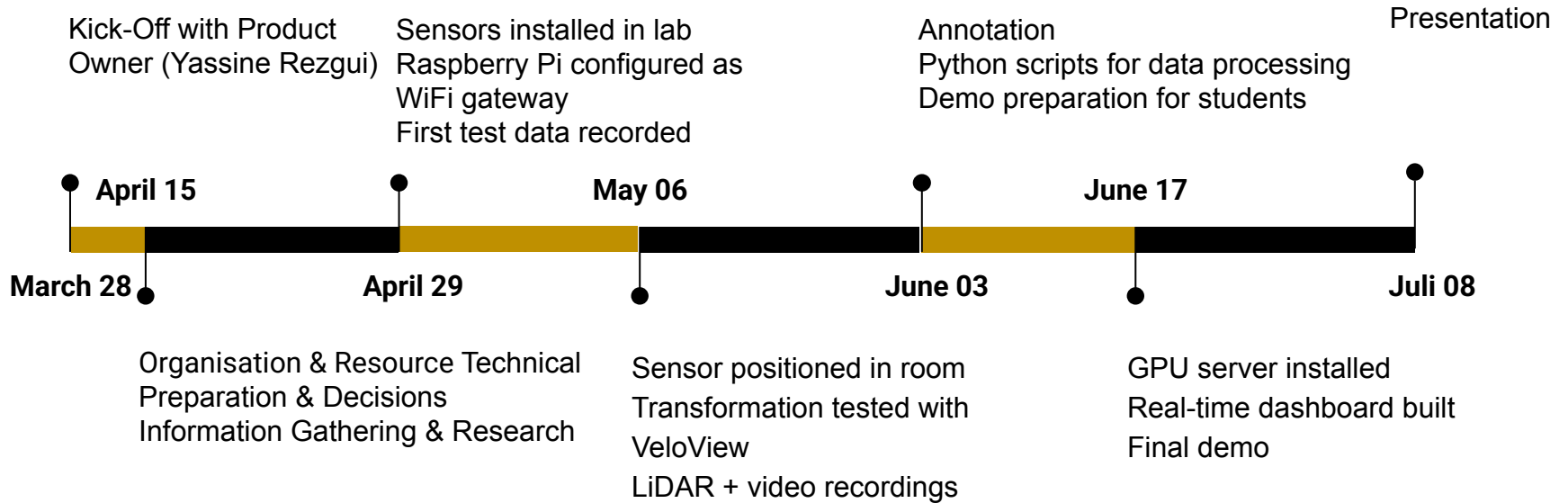
School zone safety system via outdoor LiDAR

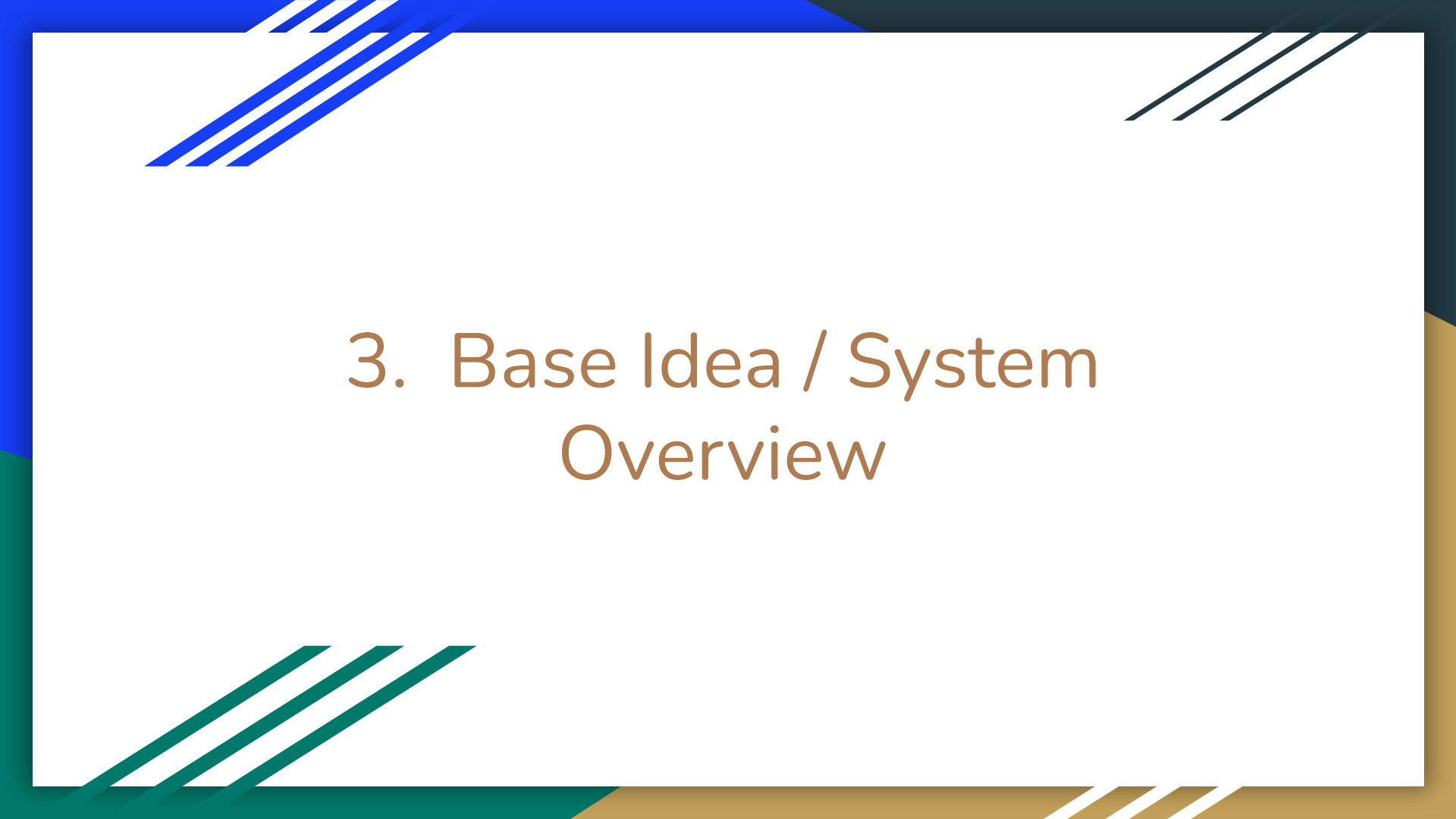
Our project:

Small indoor version of 5Safe to detect/count people in a room

2. Timeline

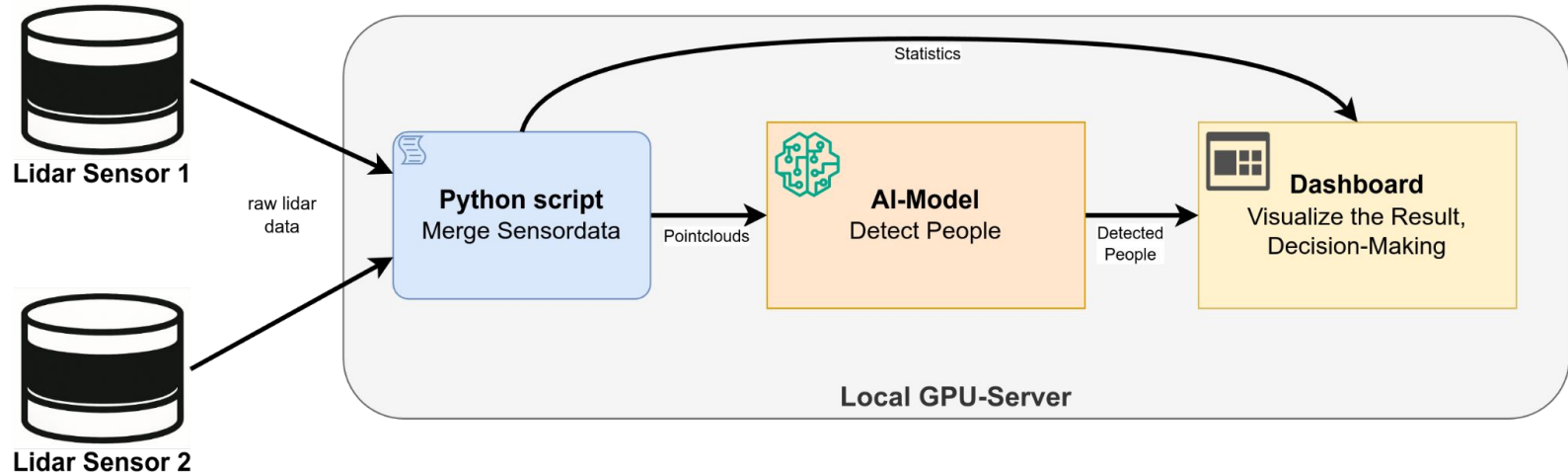
Timeline





3. Base Idea / System Overview

Concept and System Overview



Our Idea was to merge the data of 2 sensors into one Point cloud first, then pass it to an AI-Model. The python script and the AI-Model should then feed the dashboard with data

4. Hardware Setup

Sensors, Server, Network



4.1 Lidar Sensors



Available Velodyne Lidar Sensor



The Sensor used is the actual Sensor used in the official 5-Safe project. The sensor has 360° Horizontal View and 40° vertical Field of View: (-25° to +15°).

Data is outputted through a constant UDP Stream to a configurable IP-Address.



Sensor placement

Sensor 1 was placed to have the highest resolution in the center of the Room.

Sensor 2 was placed to have the highest resolution at the door.

Sensor 1 is tilted downward by 8° and Sensor 2 is tilted downward by 15° .

The Idea behind the placement of the second sensor, was to optimize the Algorithm which counted people leaving and entering the room. This Algorithm proved to be very difficult to implement, but the placement was kept as is.



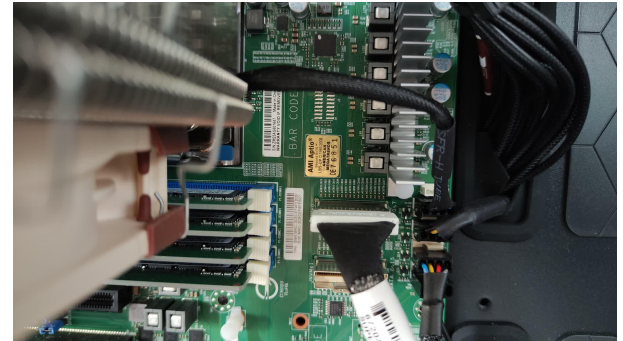


4.2 GPU-Server

GPU-Server



The Server has two NVIDIA 4090 GPU's, a 64 thread AMD CPU and 128 GB of RAM. It is currently using a Ubuntu operating System on which our Python script and its rudimentary AI implementation are running.



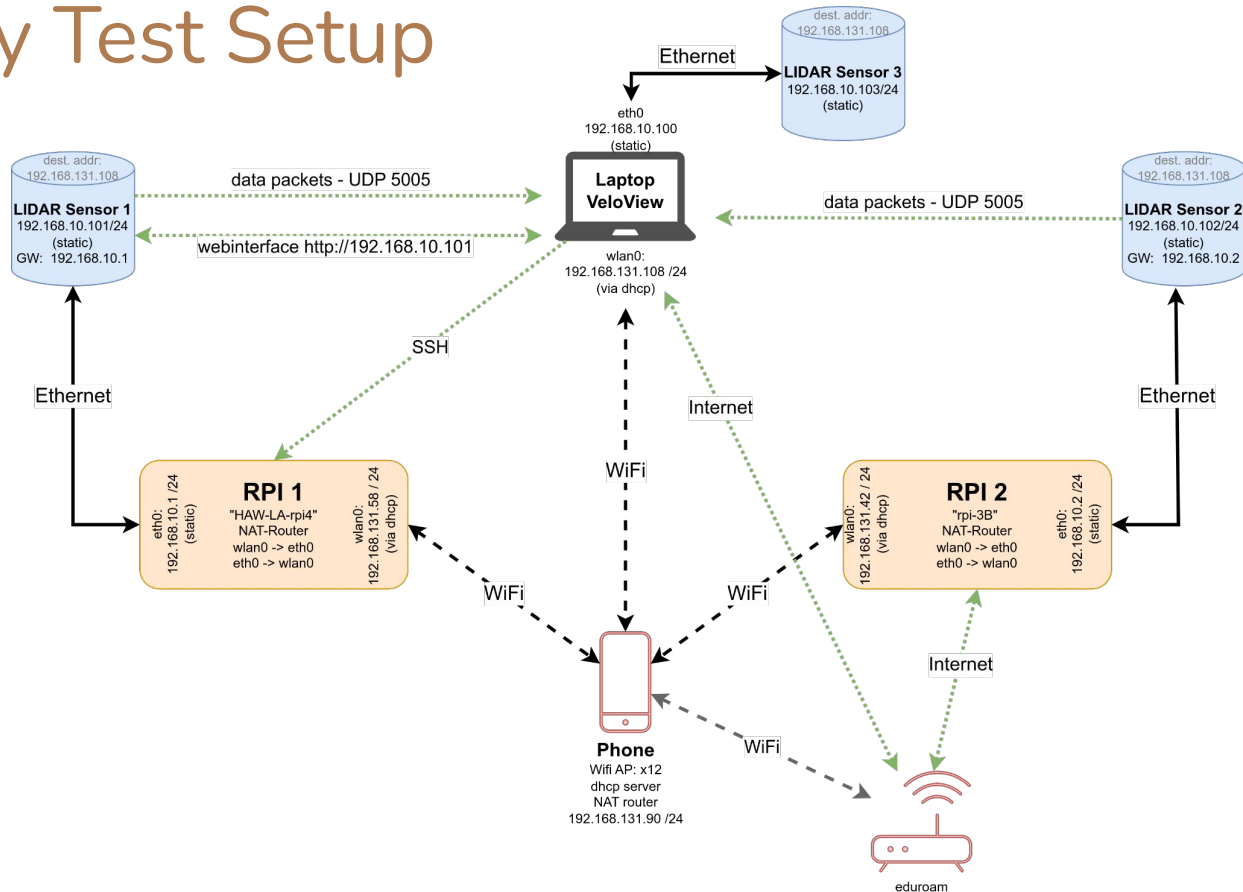


4.3 Network

Network: Early Test Setup

Initially, we used Raspberry Pi's as Ethernet–Wi-Fi gateway at the sensors.

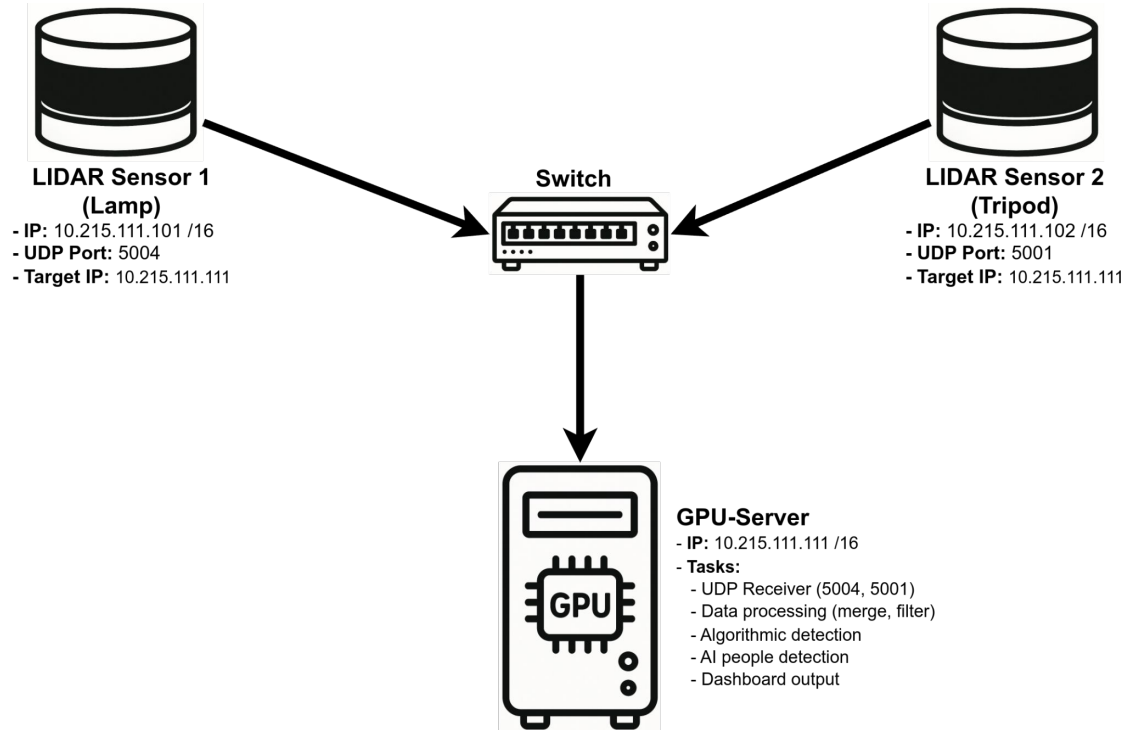
This enabled wireless sensor placement for flexible indoor testing



Network: Final Simplified Setup

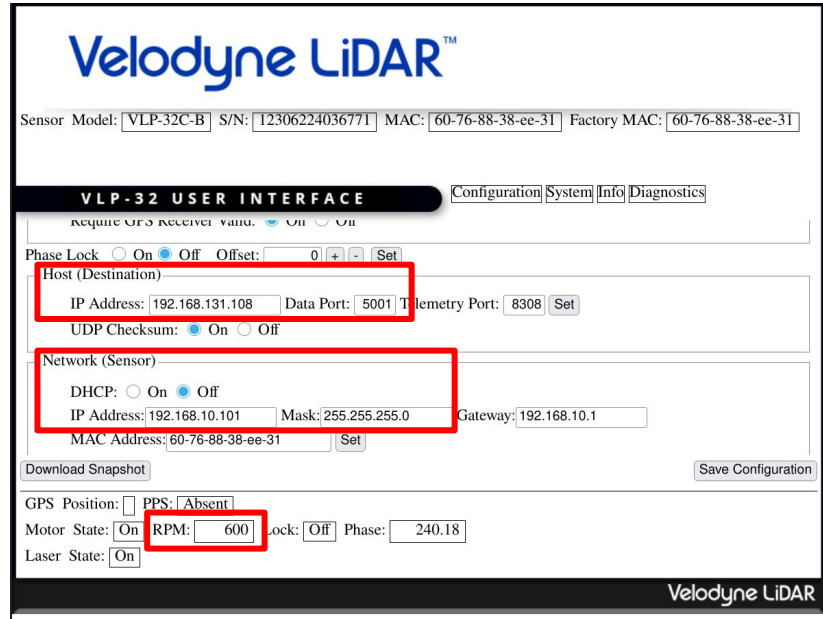
With the sensor location fixed, the Wi-Fi bridge was no longer needed.

The system now uses the room's existing LAN infrastructure.



Network: Sensor Setup & Integration

- **Sensor IP** was identified using Wireshark
- Laptop network was reconfigured to access the **web interface**
- Custom **static IP** and **destination** (GPU server) were set
- **Rotation speed** was reduced to the minimum (**300 RPM**) for 5 fps



Velodyne LiDAR™

Sensor Model: [VLP-32C-B] S/N: [12306224036771] MAC: [60-76-88-38-ee-31] Factory MAC: [60-76-88-38-ee-31]

VLP-32 USER INTERFACE Configuration System Info Diagnostics

Require GPS Receiver valid. ☒ On ☐ Off

Phase Lock ☐ On ☒ Off Offset: [0] [Set]

Host (Destination)

IP Address: [192.168.131.108] Data Port: [5001] Telemetry Port: [8308] [Set]

UDP Checksum: ☒ On ☐ Off

Network (Sensor)

DHCP: ☐ On ☒ Off

IP Address: [192.168.10.101] Mask: [255.255.255.0] Gateway: [192.168.10.1]

MAC Address: [60-76-88-38-ee-31] [Set]

Download Snapshot [Save Configuration]

GPS Position: ☐ PPS: [Absent]

Motor State: [On] RPM: [600] Lock: [Off] Phase: [240.18]

Laser State: [On]

Velodyne LiDAR

5. First Tests

Initial Tests with Sensor and VeloView

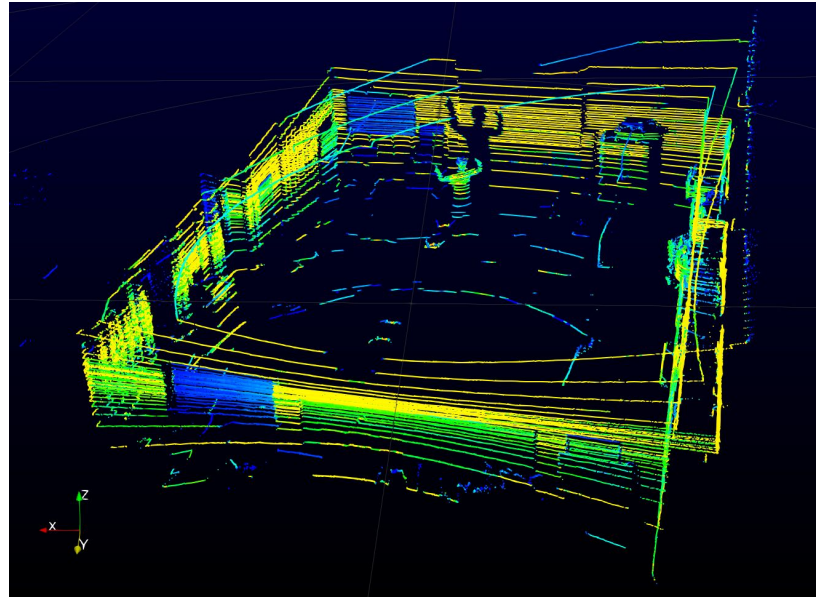
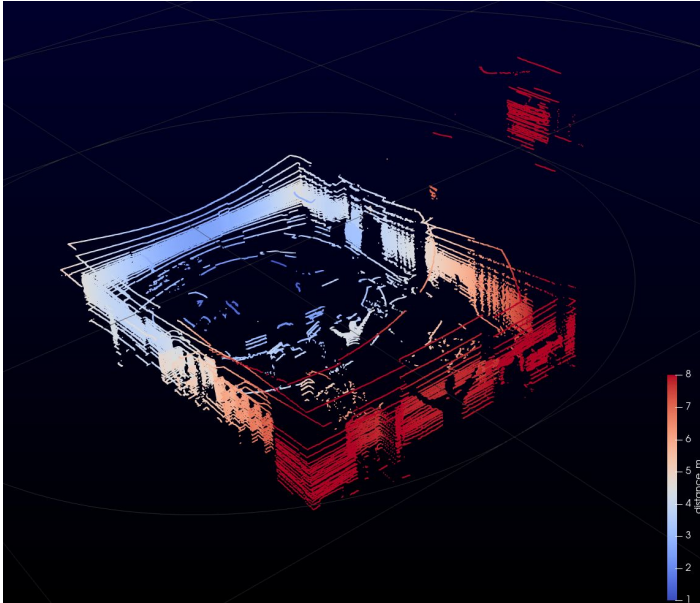
Test Setup



In the Test setup one Sensor was used.

- Sensor sent data directly to Laptop through direct LAN connection
- Laptop visualized data through VeloView
- To remotely work on the project, wireshark was used to record the stream of UDP packages into a .pcap file.

VeloView



First Tests with Veloview, showing Points colored based on distance and based on intensity.



6. Development Workflow & Tools

Codebase Overview (Git Repository)

- Git repository hosted on **GitHub**
- Contains **Python core project**, test **recordings**, helper tools
- Git LFS used to store large LiDAR recordings (.pcap, .mp4)
- Detailed **README** with usage instructions and setup steps

The screenshot shows the GitHub repository page for **HAW-LA_5Safe-LidarDetection**. The repository is public and has 9 branches and 5 tags. The main branch is **dev**. The repository description is: "5Safe Lidar Detection" developed as part of the IoT-Project module at HAW Landshut. The repository contains a **README** file, which is currently selected. The README title is **5Safe Lidar Room-Scale LiDAR People Detection – HAW-LA**. The README content states: "This repository contains the full system for 5Safe Room-Scale LiDAR Detection, developed in the IoT Project module at HAW Landshut." The repository also has a **CONTRIBUTORS** section with 2 contributors: Jonny999999 and skneitel. The **LANGUAGES** section shows the following distribution: Python 97.2% and Shell 2.8%.

File	Commit Message	Time
ai-detection	detect method now returns human points	5 days ago
dashboard	Major change in project structure, Rename several files	3 days ago
data	Major change in project structure, Rename several files	3 days ago
doc	Optimize Readme: Diagrams, fixes, details; Disable beepin...	9 hours ago
legacy_projects	Major change in project structure, Rename several files	3 days ago
lidar_system	Merge branch 'use-changed-points' into dev - New experim...	7 hours ago
scripts	Rework Readme: Add detailed instructions, diagrams, scree...	2 days ago
.gitattributes	Delete pcap.gz files, Update gitattributes so pcap.gz files ar...	2 months ago
.gitignore	File export in laz format and restructured output	last month
README.md	Optimize Readme: Diagrams, fixes, details; Disable beepin...	9 hours ago

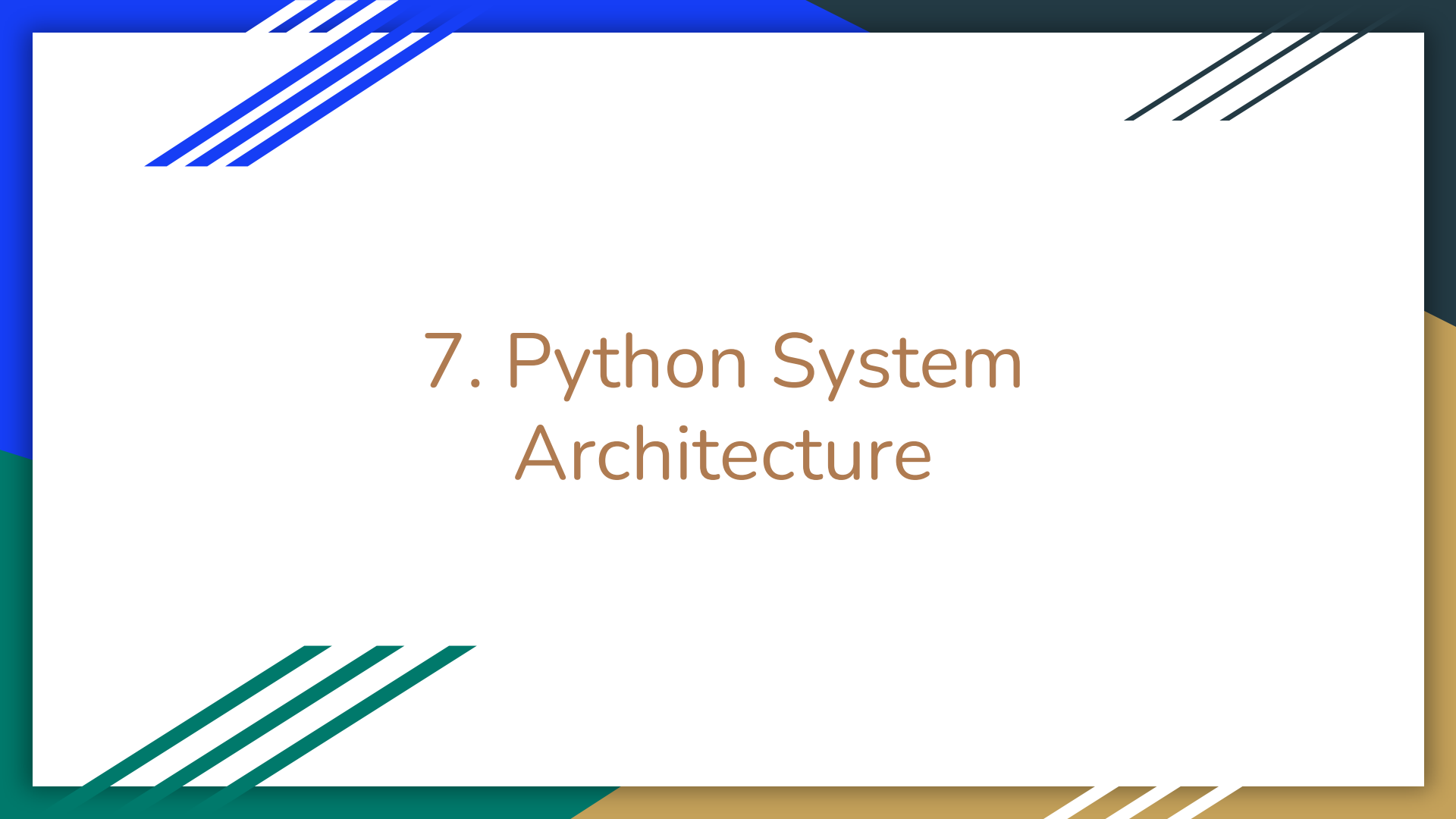
Link to Repo: https://github.com/Jonny999999/HAW-LA_5Safe-LidarDetection

Version Control & Development History

- Developed collaboratively over **7 weeks**
- **100+ commits** across **8 branches**
- Modular development using Git and **VS Code**
- Separate **branches** for testing, new features and experiments. Merged back into main when stable and tested
- **Tags** when reaching milestones

Link to Repo: https://github.com/Jonny999999/HAW-LA_5Safe-LidarDetection





7. Python System Architecture

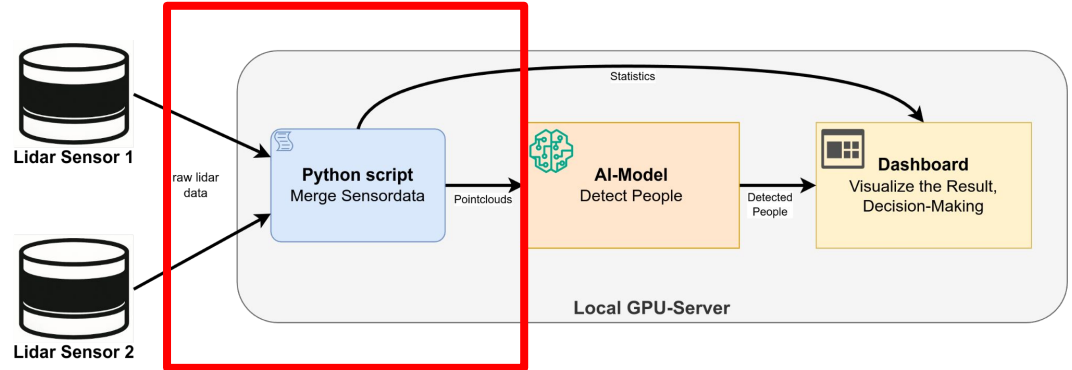
Python script: Goal & Features

Initial Features:

- **Receive** sensor data
- **Decode** to point clouds
- Transform + **merge**
- Forward **point clouds** to model
- File **export** for annotation

Extended over time:

- **Motion** detection
- **Tracking** algorithm
- **Statistics** output
- TCP socket for **dashboard**
- Independent **AI integration**

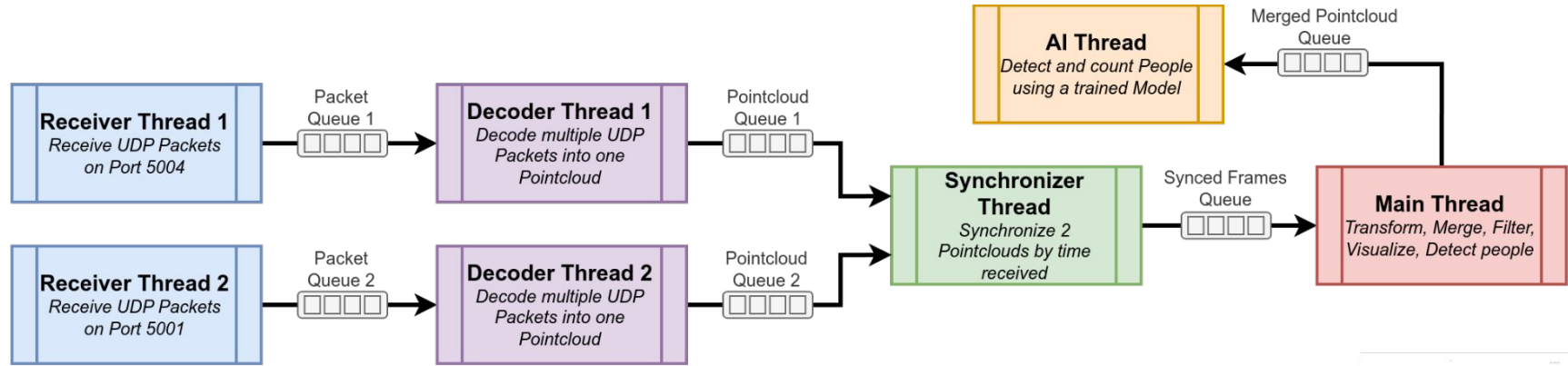




7.1 Thread Overview

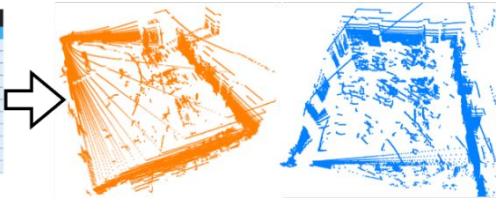


Python script: Thread Overview

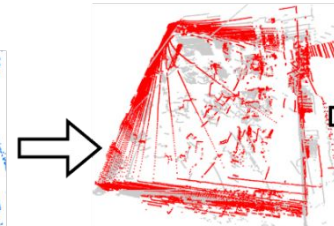


No.	Source	Destination	Protocol	Info
1	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
2	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
3	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
4	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
5	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
6	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
7	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
8	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
9	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
10	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
11	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206
12	192.168.0.89	192.168.0.2	UDP	2368 - 5004 Len=1206

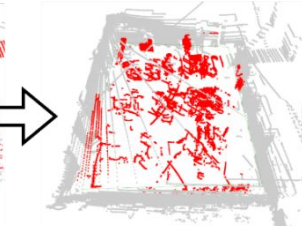
Received UDP Packets
From each Sensor



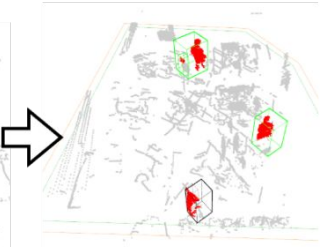
Decoded Pointclouds Sensor 1 and Sensor 2



Synced, Transformed
and Merged Pointcloud

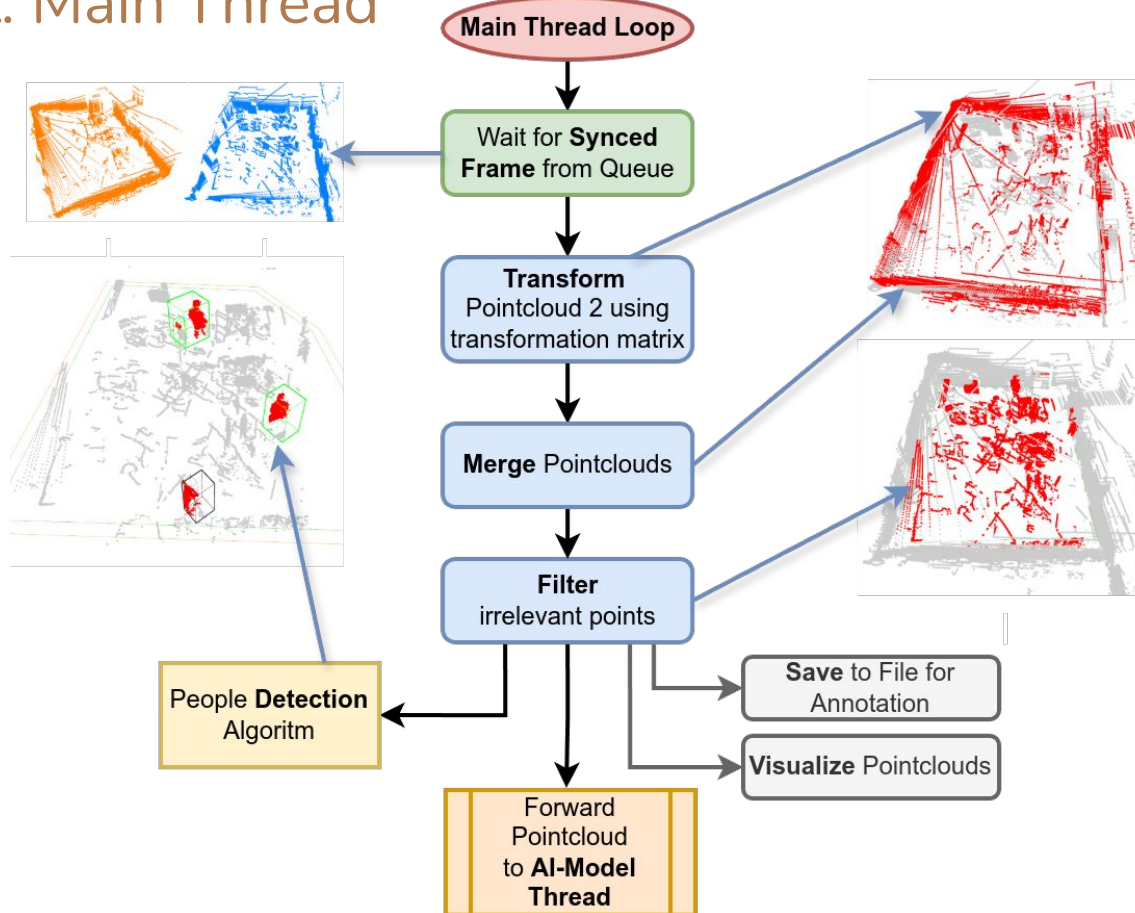


Filtering irrelevant
Points



Motion detection and
Person tracking

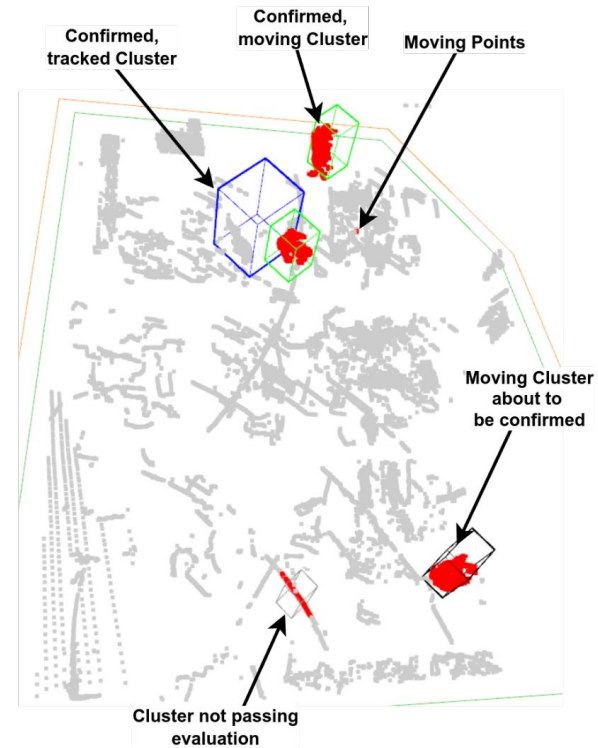
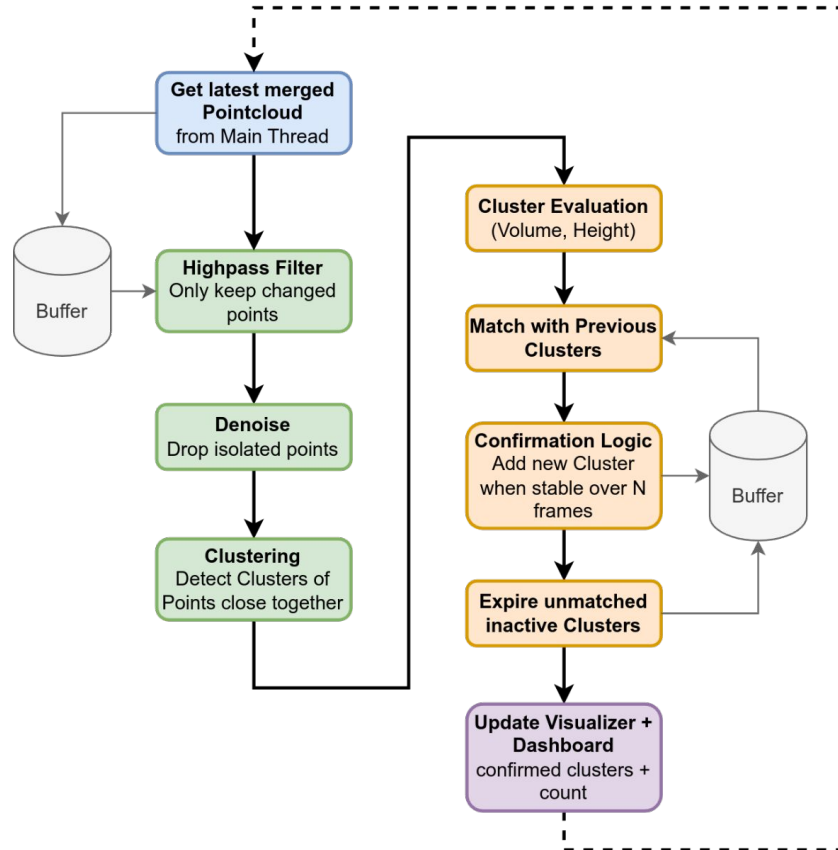
Python script: Main Thread





7.2 Tracking Algorithm

Detection / Tracking Algorithm



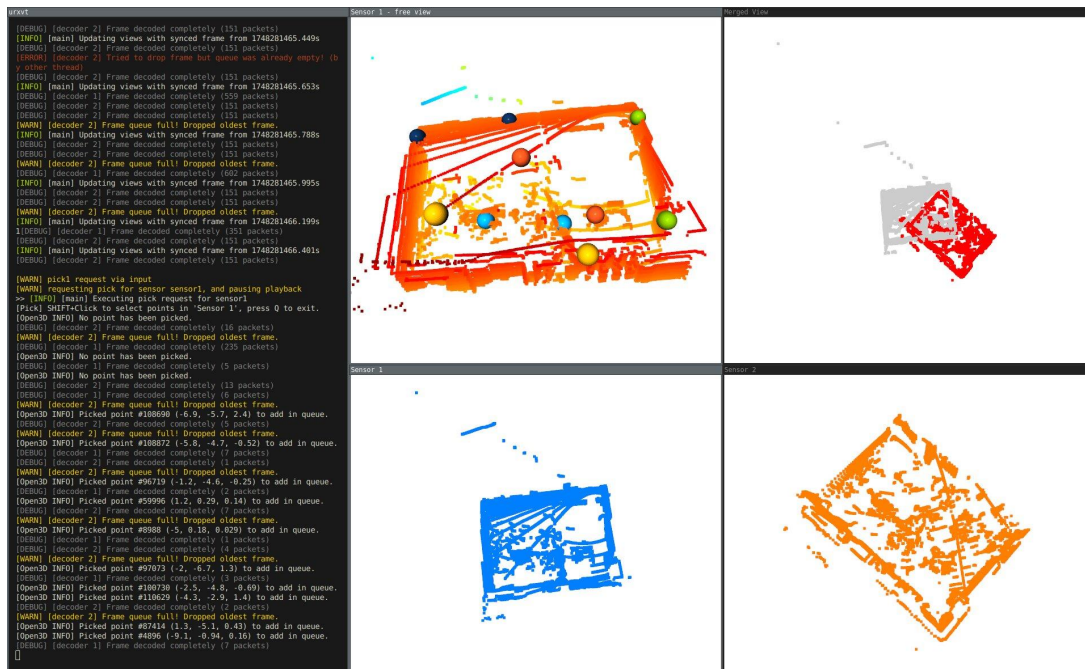


7.3 Transformation Matrix



Transformation matrix process

- To merge the data, a Transformation Matrix had to be determined
- this was done through a small external python script.
- At least three points in the room had to have their coordinates recorded for both Sensors.
- The script outputs a matrix which is used inside the main script





7.4 AI-Model Integration



Quick PoC integration of an ai-model into python script

- A simple AI Model was implemented through the use of Google Colab
- The AI Model in question is a very simple PointNet Style Model
- For training data see chapter “Annotation”
- Model decides if a point is part of a human or not (point per point basis)
- An algorithm then clusters points to determine how many people are in the room
- Implementation has weaknesses:
 - spots it was not trained in
 - people that are too close together
 - changes in room (for example an open door)



```
[ ] # 3. Modell: Einfaches PointNet Light

import torch.nn as nn
import torch.nn.functional as F

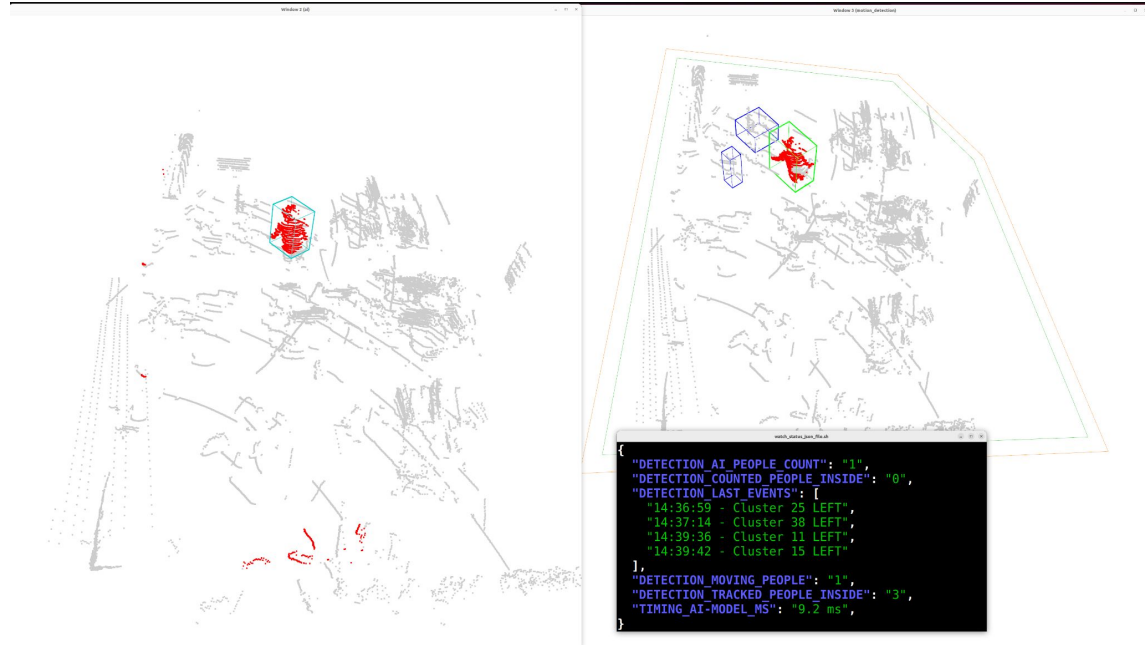
class SimplePointNet(nn.Module):
    def __init__(self, num_classes=2):
        super(SimplePointNet, self).__init__()
        self.fc1 = nn.Linear(3, 64)
        self.fc2 = nn.Linear(64, 128)
        self.fc3 = nn.Linear(128, 256)
        self.fc4 = nn.Linear(256, 128)
        self.fc5 = nn.Linear(128, num_classes)

    def forward(self, x):
        # x = [B, N, 3]
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = F.relu(self.fc4(x))
        x = self.fc5(x) # Output: [B, N, num_classes]
        return x

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = SimplePointNet().to(device)
```

Quick PoC integration of an ai-model into python script

- AI-Integration on the left.
- All red points are detected as “part of a person”
- Blue Box is the clustering algorithm



8. Annotation

Real - World LiDAR Capture

- Indoor lab setup using static LiDAR
- Captured everyday scenes: desks, chairs, boxes, **people**



Annotation Tools





Why exactly CVAT?

CVAT vs. Supervisely

Category

CVAT



Supervisely



SUPERVISELY

Licence

Free, Open Source

Freemium, Proprietary

Max. Frames

Unlimited

Limited

Supported Formats

LiDAR

LiDAR

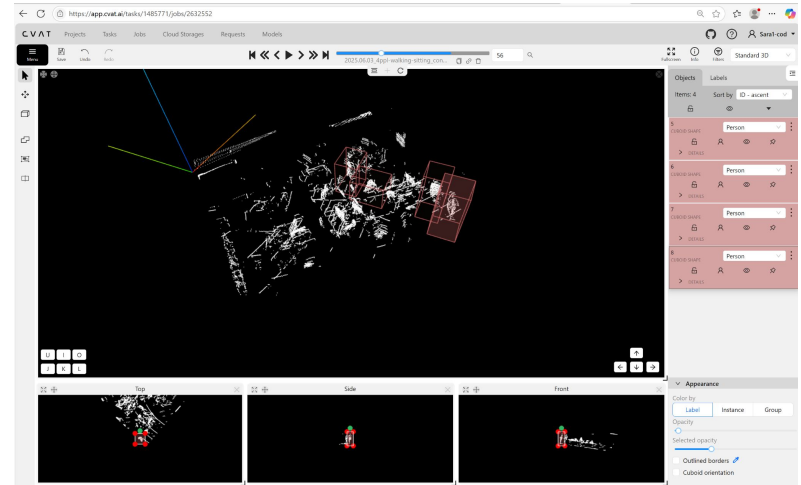
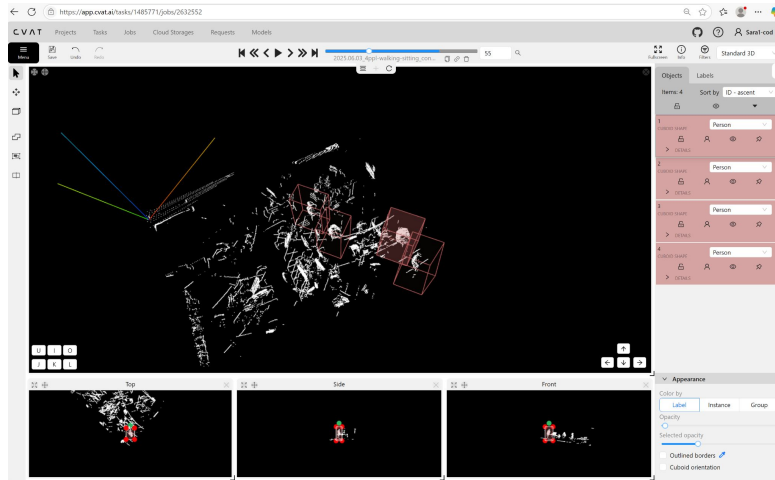
Costs

Free

Free

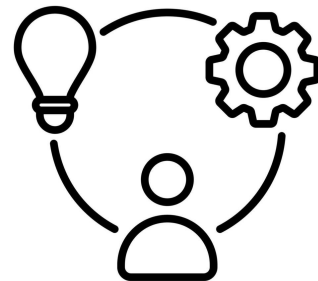
Workspace of CVAT Annotation Tool

- Each person was labeled from three different perspectives: top, side, and front
- To ensure the detection system could learn to recognize people from multiple angles



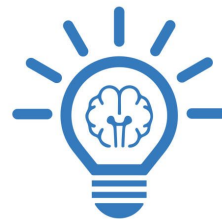
Example Annotation in JSON Format

```
1 {
2   "description": "",
3   "key": "f8768453499345cf8fdc6c1da2edfa41",
4   "tags": [],
5   "objects": [
6     {
7       "key": "0a36518f2bf44c9380a493beedb33b04",
8       "classTitle": "p1",
9       "tags": [],
10      "labelerLogin": "Sar",
11      "updatedAt": "2025-06-15T10:45:18.781Z",
12      "createdAt": "2025-06-15T10:45:18.781Z"
13    },
14    {
15      "key": "4e44dbadac0943b7a3379522bfb3aa6b",
16      "classTitle": "p1",
17      "tags": [],
18      "labelerLogin": "Sar",
19      "updatedAt": "2025-06-15T10:46:56.400Z",
20      "createdAt": "2025-06-15T10:46:56.400Z"
21    },
22    {
23      "key": "3eb523d5763142b99c62fc5ece06cee3",
24      "classTitle": "p1",
25      "tags": [],
26      "labelerLogin": "Sar",
27      "updatedAt": "2025-06-15T10:47:02.641Z",
28      "createdAt": "2025-06-15T10:47:02.641Z"
29    },
30    {
31      "key": "6a3481830afc45d5a088cf653680dbd4",
32      "classTitle": "p1",
33      "tags": [],
34      "labelerLogin": "Sar",
35      "updatedAt": "2025-06-15T10:47:07.436Z",
36      "createdAt": "2025-06-15T10:47:07.436Z"
37    },
38    {
39      "key": "fd880fa96548426cbccde97012369c72",
40      "classTitle": "p1",
41      "tags": [],
42      "labelerLogin": "Sar",
43      "updatedAt": "2025-06-15T10:47:13.618Z",
44      "createdAt": "2025-06-15T10:47:13.618Z"
45    },
46    {
47      "key": "6967b7f8675d446db9a3b8ac727cc7fd",
48      "classTitle": "p1",
49      "tags": [],
50      "labelerLogin": "Sar",
51      "updatedAt": "2025-06-15T10:47:18.936Z",
52      "createdAt": "2025-06-15T10:47:18.936Z"
53    }
54  ],
55 }
```



Annotation Challenges

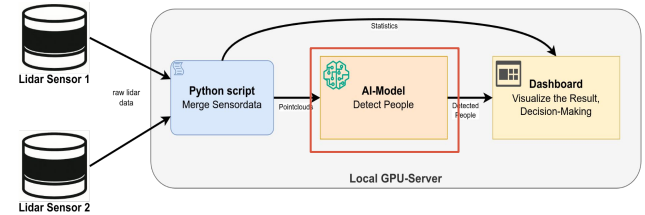
- Indoor LiDAR data is often sparse and noisy, making object boundaries unclear
- It's difficult to precisely place 3D bounding boxes without clear object outlines
- Manual Workload - It takes significant time and required extra quality control



Lesson learned

- Good tools for 3D annotation exist, but they need consistent file naming
- detailed quality control
- Some errors only become visible during model training

9. AI Model Training

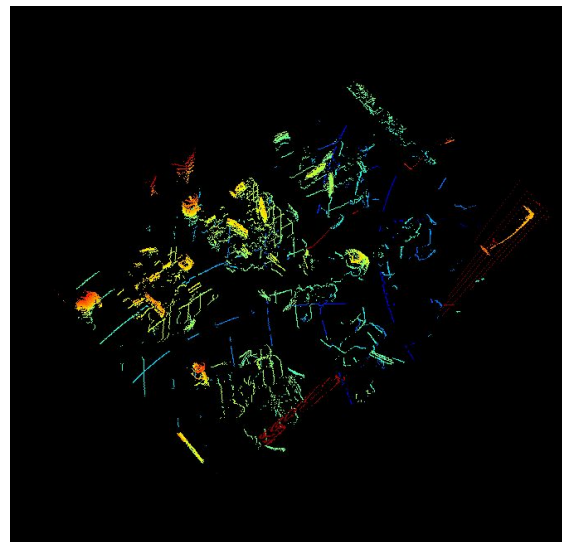
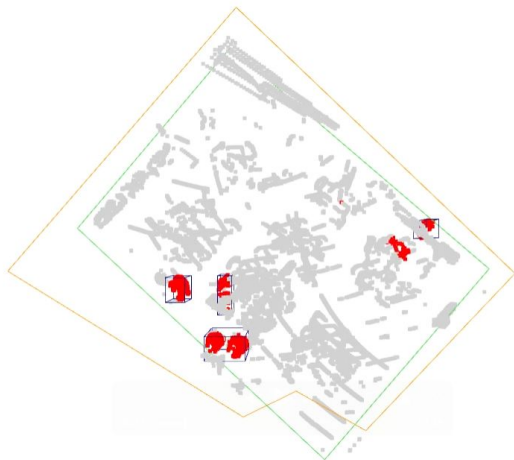


Overview

- Goal: 3D object detection on real indoor LiDAR data
- Model: TR3D from MMDetection3D
- Tools: CVAT (annotation), MMDetection3D (model), LiDAR sensor
- Focus: Data pipeline, training challenges, results

Real-World Data Collection

- Indoor lab setup using static LiDAR
- Captured everyday scenes: desks, chairs, boxes, **people**



MMDetection3D - The Detection Framework

- Open-source 3D object detection toolbox built on top of MMDetection3D and PyTorch
- Developed by the OpenMMLab team
- Supports LiDAR, RGB, multi-modality, and indoor/outdoor 3D object detection

open-mmlab/
mmdetection3d



OpenMMLab's next-generation platform for general 3D object detection.

104

Contributors

555

Issues

142

Discussions

6k

Stars

2k

Forks



Why did we choose MMDetection3D?

Feature	Why it mattered for us?
→ Indoor dataset support	Works with ScanNet-format data
→ Transformer models	Includes TR3D for cutting-edge accuracy
→ Pretrained models	Speeds up experimentation

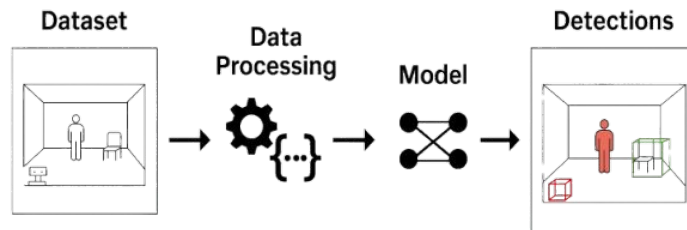
Model Choice -TR3D

- Based on transformers for indoor 3D detection
- Supports voxel-based input, high accuracy on ScanNet

Why TR3D?

- Strong performance on indoor datasets
- Transformer architecture

TR3D – LiDAR Indoor 3D Object Detection



Initial Training (Local)

- Environment: PyTorch, MMCV, MMEEngine, CUDA

What we tried:

- Converted data to ScanNet format (Python)
- Modified TR3D config
- Matched Dependency versions carefully

What failed:

- Compatibility issues between PyTorch, MMCV, and CUDA.
- Local GPU memory was not sufficient to train TR3D efficiently.
- Extremely long training times and frequent out-of-memory crashes.

```
1 import os
2 import json
3 import numpy as np
4
5 # Paths
6 cvat_json_path = 'annotations/annotations.json'
7 pointcloud_dir = 'pointclouds/' # e.g., raw .bin or .pcd files
8 output_dir = 'data/scans/scene0000_00/'
9
10 os.makedirs(os.path.join(output_dir, 'labels'), exist_ok=True)
11 os.makedirs(os.path.join(output_dir, 'pointclouds'), exist_ok=True)
12
13 # Class map: change this to your dataset classes
14 CLASS_MAPPING = {
15     "person": 0,
16     "chair": 1,
17     "table": 2
18 }
19
20 def load_cvat_json(path):
21     with open(path, 'r') as f:
22         return json.load(f)
23
24 def write_label_file(objects, frame_id, label_dir):
25     label_path = os.path.join(label_dir, f"({frame_id:05d}).txt")
26     with open(label_path, 'w') as f:
27         for obj in objects:
28             class_name = obj['label']
29             pos = obj['position']
30             dims = obj['dimensions']
31             yaw = obj.get('rotation', [0, 0, 0])[2] # Use Z angle
32
33             f.write(f"({class_name}) {pos[0]:.2f} {pos[1]:.2f} {pos[2]:.2f} "
34                   f"({dims[0]:.2f} {dims[1]:.2f} {dims[2]:.2f}) "
35                   f"({yaw:.2f})\n")
36
37 def convert_annotations(cvat_data):
38     for item in cvat_data['frames']:
39         frame_id = item['frame']
40         filename = item['pointcloud']['path'] # raw point cloud file path
41         objects = item['objects']
42
43         # Copy or move pointcloud file
44         src_pc_path = os.path.join(pointcloud_dir, filename)
45         dst_pc_path = os.path.join(output_dir, 'pointclouds', f"({frame_id:05d}).bin")
46
47         if os.path.exists(src_pc_path):
48             os.system(f"cp {src_pc_path} {dst_pc_path}")
49         else:
50             print(f"Missing point cloud: {src_pc_path}")
51
52         # Write annotation
53         write_label_file(objects, frame_id, os.path.join(output_dir, 'labels'))
54
55 # Load CVAT JSON and convert
56 cvat_data = load_cvat_json(cvat_json_path)
57 convert_annotations(cvat_data)
58
59 # Optional: write meta/class map
60 meta_dir = 'data/meta_data'
61 os.makedirs(meta_dir, exist_ok=True)
62 with open(os.path.join(meta_dir, 'class_mapping.txt'), 'w') as f:
63     for cls in CLASS_MAPPING:
64         f.write(f"({cls}) {CLASS_MAPPING[cls]}\n")
65
66 print("✅ Conversion to ScanNet-style format complete.")
```

Google Colab Attempt



- Cloned the MMDetection3D GitHub repository. (Runtime: T4 GPU)
- Uploaded the preprocessed dataset to Google Drive and mounted it in Colab.
- Attempted to run training using the previously created TR3D configuration.

A screenshot of the Google Colab web interface. The top bar shows the Colab logo, a star icon, and the text 'AI Model.ipynb'. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The main area is divided into three tabs: 'Commands', '+ Code', and '+ Text'. The 'Commands' tab is active, showing a list of commands. The first command is 'from google.colab import files' followed by 'uploaded = files.upload() # Upload AI_Model.zip or individual files'. Below this is a file upload widget with a 'Choose file' button and a message: 'No file chosen. Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.' The second command is '!unzip /content/AI_Model.zip -d /content/AI_Model'. Below this is a directory listing showing the contents of the '/content/AI_Model' directory. The listing shows a hierarchy of folders and files, including 'AI_Model', 'AI_Model/MACOSX/.AI_Model', 'AI_Model/mmdetection3d', 'AI_Model/mmdetection3d/demo', 'AI_Model/mmdetection3d/tools', 'AI_Model/mmdetection3d/docker', 'AI_Model/mmdetection3d/LICENSE', 'AI_Model/mmdetection3d/requirements.txt', 'AI_Model/mmdetection3d/.pre-commit-config-zh-cn.yaml', 'AI_Model/mmdetection3d/.pre-commit-config.yaml', 'AI_Model/mmdetection3d/CITATION.cff', 'AI_Model/mmdetection3d/projects', 'AI_Model/mmdetection3d/resources', 'AI_Model/mmdetection3d/tests', 'AI_Model/mmdetection3d/MANIFEST.in', 'AI_Model/mmdetection3d/requirements', 'AI_Model/mmdetection3d/docs', 'AI_Model/mmdetection3d/README_zh-CN.md', 'AI_Model/mmdetection3d/README.md', 'AI_Model/mmdetection3d/readthedocs.yml', 'AI_Model/mmdetection3d/mmdet3d', 'AI_Model/mmdetection3d/README.md', and 'AI_Model/mmdetection3d/setup.py'.

```
from google.colab import files
uploaded = files.upload() # Upload AI_Model.zip or individual files

!unzip /content/AI_Model.zip -d /content/AI_Model
```

Archives: /content/AI_Model.zip
creating: /content/AI_Model/AI_Model/
inflating: /content/AI_Model/MACOSX/.AI_Model
inflating: /content/AI_Model/mmdetection3d/convert_supervisory_to_kitti.py
creating: /content/AI_Model/mmdetection3d/
creating: /content/AI_Model/mmdetection3d/supervisory_raw/
creating: /content/AI_Model/mmdetection3d/data/
creating: /content/AI_Model/mmdetection3d/demo/
creating: /content/AI_Model/mmdetection3d/tools/
creating: /content/AI_Model/mmdetection3d/docker/
inflating: /content/AI_Model/mmdetection3d/LICENSE
inflating: /content/AI_Model/mmdetection3d/requirements.txt
inflating: /content/AI_Model/mmdetection3d/.pre-commit-config-zh-cn.yaml
inflating: /content/AI_Model/mmdetection3d/.pre-commit-config.yaml
inflating: /content/AI_Model/mmdetection3d/CITATION.cff
creating: /content/AI_Model/mmdetection3d/projects/
creating: /content/AI_Model/mmdetection3d/resources/
creating: /content/AI_Model/mmdetection3d/tests/
inflating: /content/AI_Model/mmdetection3d/MANIFEST.in
creating: /content/AI_Model/mmdetection3d/requirements/
creating: /content/AI_Model/mmdetection3d/docs/
inflating: /content/AI_Model/mmdetection3d/README_zh-CN.md
inflating: /content/AI_Model/mmdetection3d/README.md
creating: /content/AI_Model/mmdetection3d/readthedocs.yml
creating: /content/AI_Model/mmdetection3d/mmdet3d/
inflating: /content/AI_Model/mmdetection3d/README.md
inflating: /content/AI_Model/mmdetection3d/setup.py

Google Colab Attempt Issues



- Frequent dependency conflicts (e.g., mmcv vs. mmengine vs. torch versions).
- CUDA mismatch, compile errors
- Training failed mid-way or never started

```
import torch, mmcv, mmengine, mmdet, mmdet3d
print(torch.__version__, mmcv.__version__, mmengine.__version__, mmdet.__version__, mmdet3d.__version__, torch.cuda.is_available())

Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cu118
Collecting torch==2.6.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torch-2.6.0%28cu118-cp311-cp311-linux_x86_64.whl.metadata (27 kB)
Collecting torchvision==0.20.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torchvision-0.20.0%28cu118-cp311-cp311-linux_x86_64.whl (6.5 MB)
Collecting torchaudio==2.6.0+cu118
  Using cached https://download.pytorch.org/whl/cu118/torchaudio-2.6.0%28cu118-cp311-cp311-linux_x86_64.whl.metadata (6.6 kB)
Collecting filelock (from torch==2.6.0+cu118)
  Using cached filelock-3.18.0-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions==4.10.0 in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (4.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/site-packages (from torch==2.6.0+cu118) (3.1.6)
Collecting fsspec (from torch==2.6.0+cu118)
  Using cached fsspec-2025.1.1-py3-none-any.whl.metadata (11 kB)
Collecting nvidia-cuda-nvrtc-cu11==11.8.89 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_nvrtc_cu11-11.8.89-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia_cuda_runtime_cu11==11.8.89 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_runtime_cu11-11.8.89-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia_cuda_cupti_cu11==11.8.87 (from torch==2.6.0+cu118)
  Using cached nvidia_cuda_cupti_cu11-11.8.87-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu11==11.8.89 (from torch==2.6.0+cu118)
  Using cached nvidia_cudnn-cu11-11.8.89-py3-none-manylinux2014_x86_64.whl (663.9 MB)
Collecting nvidia-cublas-cu11==11.11.3.6 (from torch==2.6.0+cu118)
  Using cached nvidia_cublas-cu11-11.11.3.6-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu11==10.9.0.58 (from torch==2.6.0+cu118)
  Using cached nvidia_cufft-cu11-10.9.0.58-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu11==10.3.0.86 (from torch==2.6.0+cu118)
  Using cached nvidia_curand-cu11-10.3.0.86-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu11==11.4.1.48 (from torch==2.6.0+cu118)
  Using cached nvidia_cusolver-cu11-11.4.1.48-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparselt-cu11==11.7.5.86 (from torch==2.6.0+cu118)
  Using cached nvidia_cusparselt-cu11-11.7.5.86-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-nccl-cu11==2.12.5 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/cu118/nvidia_nccl_cu11-2.12.5-py3-none-manylinux2014_x86_64.whl (147.8 MB)
Collecting nvidia-nvtx-cu11==11.8.86 (from torch==2.6.0+cu118)
  Using cached nvidia_nvtx-cu11-11.8.86-py3-none-manylinux2014_x86_64.whl.metadata (1.7 kB)
Collecting triton==3.2.0 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/triton-3.2.0-cp311-cp311-manylinux_2_28_x86_64.whl.metadata (1.4 kB)
Collecting sympy==1.13.1 (from torch==2.6.0+cu118)
  Using cached https://download.pytorch.org/whl/sympy-1.13.1-py3-none-any.whl (6.2 MB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from torchvision==0.20.0+cu118) (2.2.6)
INFO: pip is looking at multiple versions of torchvision to determine which version is compatible with other requirements. This could take a while.
ERROR: Cannot install torch==2.6.0+cu118 and torchvision==0.20.0+cu118 because these package versions have conflicting dependencies.

The conflict is caused by:
  The user requested torch==2.6.0+cu118
  torchvision 0.20.0+cu118 depends on torch==2.5.0
```



GPU Server Attempt

- Ubuntu with Python 3.10, CUDA 11.3, and the exact required versions of PyTorch and MMCV
- Set up the MMDetection3D framework from source
- Uploaded the dataset locally to the server for faster I/O
- Launched the TR3D training pipeline using the same config used previously

- [illegible]

```

note: This error originates from a subprocess, and is likely not a problem with pip.
Traceback (most recent call last):
  File <string>, line 2, in <module>
  File <pip-setuptools-caller>, line 36, in <module>
  File /home/maible/anaconda3/setup.py, line 192, in <module>
    setup()
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/_init___py, line 115, in setup
    distutils.core.setup(**attrs)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/distutils/core.py, line 186, in setup
    return run_commands(dist)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/distutils/core.py, line 282, in run_commands
    run_command(dist)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/distutils/dist.py, line 1062, in run_commands
    self.run_command(cmd)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/dist.py, line 1182, in run_command
    self.run_command(cmd)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/distutils/dist.py, line 1021, in run_command
    cmd_obj.run()
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/site-packages/setuptools/command/develop.py, line 39, in run
    subprocess.check_call(cmd)
  File /home/maible/anaconda3/envs/mmdet3d/lib/python3.9/subprocess.py, line 373, in check_call
    raise CalledProcessError(retcode, cmd)
subprocess.CalledProcessError: Command '['/home/maible/anaconda3/envs/mmdet3d/bin/python3.9', '-m', 'pip', 'install', '-e', '.', '--user-pip52', '-no-deps']' returned non-zero status 1.

[end of output]

```



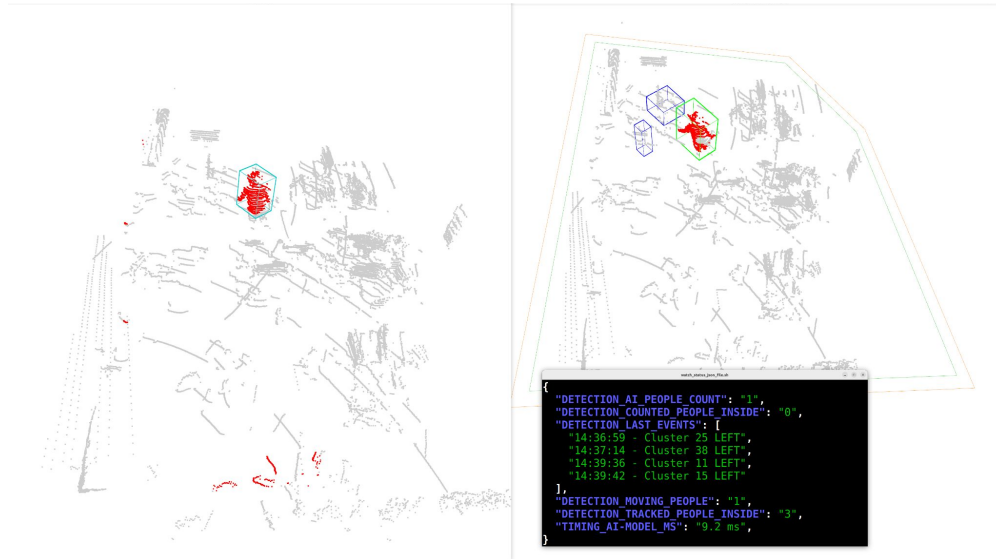
Results & Observations

- ✓ Successfully annotated and formatted a real indoor LiDAR dataset.
- ✓ Verified that data pipelines and configurations were properly structured.
- ✓ Ran small inference/ debug sessions to confirm compatibility.
- ✗ No full training run of TR3D completed successfully on any platform.
- ✗ The local PC lacked memory and compute power.
- ✗ Colab & GPU Server were unstable and hard to control.

Final Decision-making GUI/App

No model predictions due to repeated training failures → nothing to visualize

BUT therefore, we have the quick PoC integration of an AI-Model into Python Script shown earlier





Known Limitations/ Lessons learned

- TR3D requires substantial computation and a very specific environment
- MMDetection3D is highly sensitive to version mismatches and lacks robust error reporting
- Training a transformer model like TR3D on a small dataset may not be efficient or even feasible
- Documentation for advanced 3D models is often incomplete or out-of-date



Conclusion

This project laid the groundwork for LiDAR-based 3D objection detection in indoor environments by:

- Collecting and annotating real-world point cloud data using CVAT
- Creating a custom dataset compatible with MMDetection3D
- Attempting to train the TR3D model across multiple platforms (local PC, Google Colab, Server)

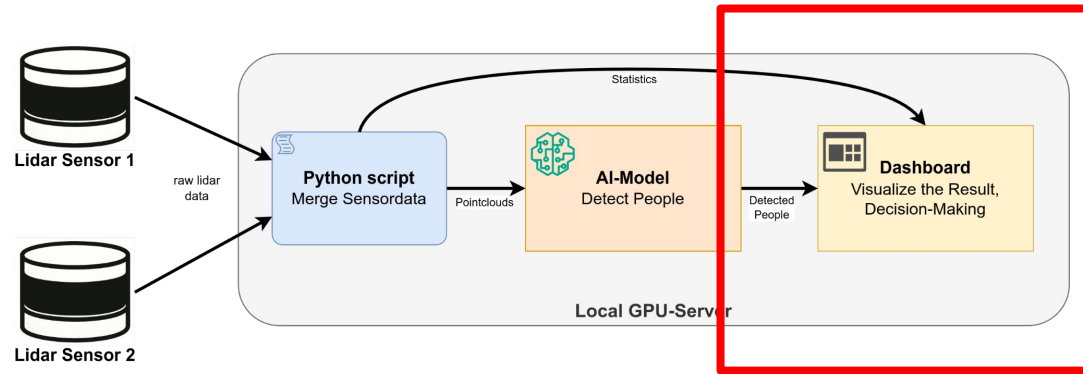


Recommendations

- Trying simpler models before using transformer-based models like TR3D
- Using Docker containers or Conda with versions for reproducibility
- Considering switching to other frameworks (e.g. Open3D-ML, ROS with PCL) if MMDetection3D proves too unstable

10. Dashboard

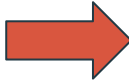
Dashboard: Goal & Features



Improvised initial CLI-based Dashboard

Visualizes selected Variables
In a separate Window

```
{
  "DETECTION_LAST_EVENTS": [
    "16:40:02 - Person LEFT",
    "16:40:06 - Person LEFT"
  ],
  "DETECTION_MOVING_PEOPLE_INSIDE": "1",
  "DETECTION_TRACKED_PEOPLE_INSIDE": "0",
  "TIMING_FRAMERATE_DECODER_1": "5.0 fps",
  "TIMING_FRAMERATE_DECODER_2": "5.3 fps",
  "TIMING_PROCESSING_DURATION_MS": "31 ms",
  "TIMING_SYNCED_FRAMERATE": "5.2 fps (193 ms)",
  "LOG_LAST_ERRORS": [
    "16:40:02 - '[track_room_occupancy] decremented 'people inside' below 0 -> clipping to 0'",
    "16:40:06 - '[track_room_occupancy] decremented 'people inside' below 0 -> clipping to 0'"
  ],
  "LOG_LAST_WARNINGS": [
    "16:39:58 - '[processor] too few frames in buffer for processing, waiting for buffer to fill up...",
    "16:39:58 - '[processor] too few frames in buffer for processing, waiting for buffer to fill up...",
    "16:39:58 - '[processor] too few frames in buffer for processing, waiting for buffer to fill up...",
    "16:40:02 - '[track_room_occupancy] Person LEFT room'",
    "16:40:06 - '[track_room_occupancy] Person LEFT room'"
  ]
}
```



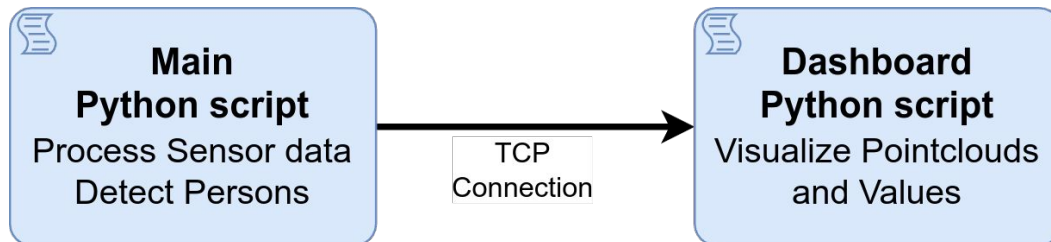
```
watch_status_json_file.sh
{
  "DETECTION_AI_PEOPLE_COUNT": "1",
  "DETECTION_COUNTED_PEOPLE_INSIDE": "0",
  "DETECTION_LAST_EVENTS": [
    "14:36:59 - Cluster 25 LEFT",
    "14:37:14 - Cluster 38 LEFT",
    "14:39:36 - Cluster 11 LEFT",
    "14:39:42 - Cluster 15 LEFT"
  ],
  "DETECTION_MOVING_PEOPLE": "1",
  "DETECTION_TRACKED_PEOPLE_INSIDE": "3",
  "TIMING_AI-MODEL_MS": "9.2 ms",
}
```



Failed TCP GUI approach

Idea:

- Dashboard should run on different device, so that result can be displayed anywhere
- Server sends data to any connecting device
- Device visualizes data





11. Project Result

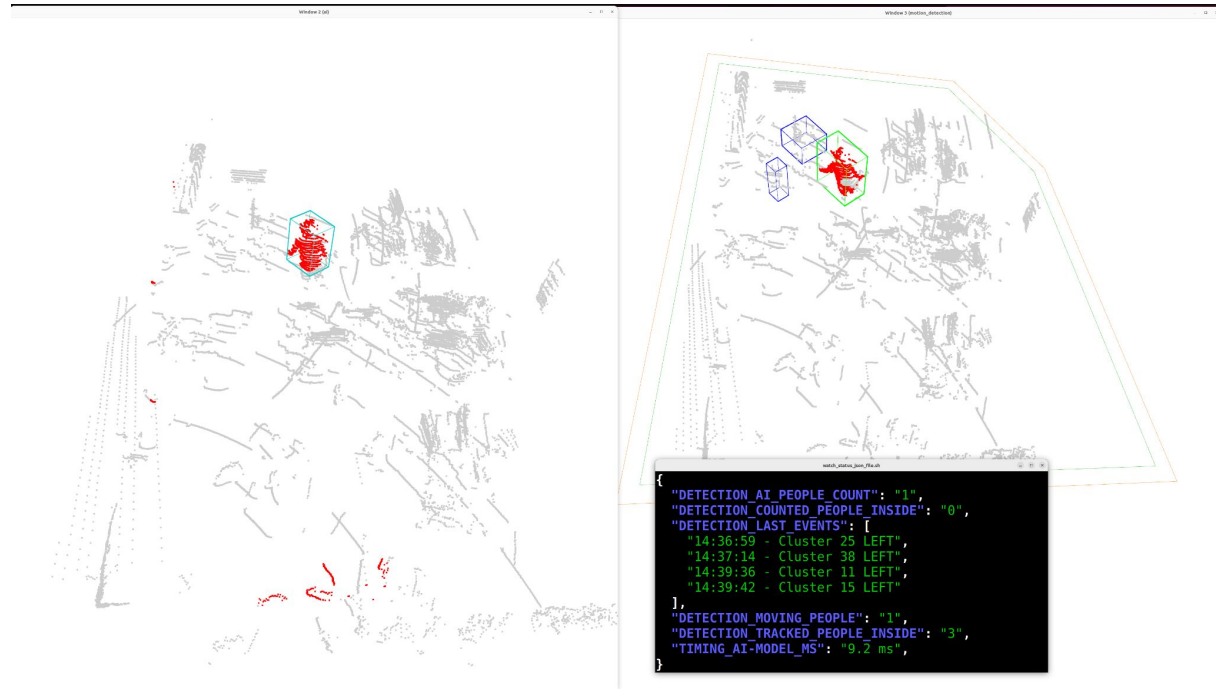
Project Result – Final Setup

- Fully working **live detection system**
- **2 LiDAR sensors** + GPU server setup
- Real-time **point cloud visualization**
- Terminal based **dashboard**
- Custom **detection algorithm** and **AI integration** tested
- AI team provided **annotated dataset** and explored **model training** → Valuable insights on toolchains, dependencies, and limitations



Project Result – Final Output

- Side-by-side AI (left) vs. Algorithm (right) detection output
- Visual debugging with bounding boxes and cluster tracking
- Status and debug info sent to dashboard for monitoring



12. Live Demo

Live Demo

1. Hardware setup
2. What's shown on the screen
3. Point cloud merging illustrated
4. People tracking - motion based
5. People tracking - change based
6. AI-Tracking
7. Comparison



13. Next Steps

Thank You

FOR YOUR ATTENTION



QUESTIONS

